

Input Format

First line starts with T, which is the number of test cases.
Each test case will contain N number of stones.

Output Format

Print "Yes" in the case Alice wins, else print "No".

Constraints

$$1 \leq T \leq 1000$$

$$1 \leq N \leq 10000$$

Sample Input and Output

Input

3

1

6

7

Output

Yes

Yes

No

Answer: (penalty regime: 0 %)

```
1 #include<stdio.h>
2 int main()
3 {
4     int T,i =0,n,t;
5     scanf("%d",&T);
6     while(i<T)
7     {
8         scanf("%d",&n);
9         t=n/4;
10        if(t%2==0 && n%2==0)
11        {
12            printf("No\n");
13        }
14        else if(t%2==1 && n%2==1)
15        {
16            printf("No\n");
17        }
18        else
19        {
20            printf("Yes\n");
21        }
22        i++;
23    }
24    return 0;
25 }
26
```

Input	Expected	Got
-------	----------	-----

3	Yes	Yes
---	-----	-----

1	Yes	Yes
---	-----	-----

6	No	No
---	----	----

7		
---	--	--

Passed all tests! ✓

the value to process.

Sample Input

630

Sample Output

2

Explanation

Add the holes count for each digit, 6, 3 and 0. Return $1 + 0 + 1 = 2$.

Sample Case 1

Sample Input

1288

Sample Output

4

Explanation

Add the holes count for each digit, 1, 2, 8, 8. Return $0 + 0 + 2 + 2 = 4$.

Answer: (penalty regime: 0 %)

```
1 #include<stdio.h>
2 int main()
3 {
4     int a,b,n = 0;
5
6     scanf("%d",&a);
7     while (a>0)
8     {
9         b=a%10;
10        if(b==0 || b==6 || b==9 || b==4)
11        {
12            n=n+1;
13        }
14        else if (b==8)
15        {
16            n=n+2;
17        }
18        a=a/10;
19    }
20    printf("%d",n);
21 }
22
23
24
25
```

	Input	Expected	Got	
✓	630	2	2	✓
✓	1288	4	4	✓

Passed all tests! ✓

Sample Output 1:

4

Sample Input 2:

5

Sample Output 2:

3

Explanation:

For test case 1, $N=10$.

According to Manish (\$1, \$2, \$3, ... \$10) must be distributed.

But as per Manisha only (\$1, \$2, \$3, \$4) coins are enough to purchase any item ranging from \$1 to \$10. Hence minimum is 4. Likewise denominations could also be (\$1, \$2, \$3, \$5). Hence answer is still 4.

For test case 2, $N=5$.

According to Manish (\$1, \$2, \$3, \$4, \$5) must be distributed.

But as per Manisha only (\$1, \$2, \$3) coins are enough to purchase any item ranging from \$1 to \$5. Hence minimum is 3. Likewise, denominations could also be (\$1, \$2, \$4). Hence answer is still 3.

Answer: (penalty regime: 0 %)

```
1 #include<stdio.h>
2 int main()
3 {
4     int n,r=0;
5     scanf("%d",&n);
6     while(n!=0)
7     {
8         n=n/2;
9         r=r+1;
10    }
11    printf("%d",r);
12 }
```

	Input	Expected	Got	
✓	10	4	4	✓
✓	5	3	3	✓
✓	20	5	5	✓
✓	500	9	9	✓
✓	1000	10	10	✓

Passed all tests! ✓

Finish review

For each test case, the first line contains an integer N, representing the number of numbers.

Each of the following N lines contains an integer.

Output Format

The output should be a single line containing the count of numbers where the numbers are odd numbers.

Output Format

The count of numbers where the numbers are odd numbers.

Example Input / Output 1

Input

5 10 15 20 25 30 35 40 45 50

Output

5

Explanation:

The numbers meeting the criteria are 5, 15, 25, 35, 45.

Answer: (penalty regime: 0 %)

```
1 #include<stdio.h>
2 int main()
3 {
4     int q,x=0;
5     while(scanf("%d",&n)!=-1)
6     {
7         if(n%2!=0)
8         {
9             x++;
10        }
11    }
12    printf("%d",x);
13 }
```

	Input	Expected	Got
✓	5 10 15 20 25 30 35 40 45 50	5	5
Passed all tests! ✓			

Given a number N, return true if and only if it is a confusing number, which satisfies the following condition:

becomes a **different** number with each digit valid.

Example 1:

6 -> 9

Input: 6

Output: true

Explanation:

We get 9 after rotating 6, 9 is a valid number and $9 \neq 6$

Example 2:

89 -> 68

Input: 89

Output: true

Explanation:

We get 68 after rotating 89, 86 is a valid number and $86 \neq 89$.

Example 3:

11 -> 11

Input: 11

Output: false

Explanation:

We get 11 after rotating 11, 11 is a valid number but the value remains the same, thus 11 is not a confusing number.

Note:

1. $0 \leq N \leq 10^9$
2. After the rotation we can ignore leading zeros, for example if after rotation we have 0008 then this number is considered as just 8.

Answer: (penalty regime: 0 %)

```
1 #include<stdio.h>
2 int main()
3 {
4     int n,x,y=1;
5     scanf("%d",&n);
6     while(n!=0 && y==1)
7     {
8         x=n%10;
9         n=n/10;
10        if(x==2 || x==3 || x==4 || x==7)
11        {
12            y++;
13        }
14    }
15    if(y==1)
16    {
17        printf("true");
18    }
19    else
20    {
21        printf("false");
22    }
23 }
```

	Input	Expected	Got	
--	-------	----------	-----	--

✓	9	true	true	✓
✓	89	true	true	✓
✓	25	false	false	✓

- Item 1 has 1 macronutrients.
- $1 + 2 \equiv 3$; observe that this is the max total, and having avoided having exactly $k = 2$ macronutrients.

Sample Input 1

2
1

Sample Output 1

2

Explanation 1

- Cannot use item 1 because $k = 1$ and $sum \equiv k$ has to be avoided at any time.
- Hence, max total is achieved by $sum = 0 + 2 = 2$.

Sample Case 2

Sample Input For Custom Testing

Sample Input 2

3
3

Sample Output 2

5

Explanation 2

$2 + 3 = 5$, is the best case for maximum nutrients.

Answer: (penalty regime: 0 %)

```

1 #include <stdio.h>
2 int main()
3 {
4     long int n,k,i,sum=0;
5     scanf("%ld%ld",&n,&k);
6     for (i=1;i<=n;i++)
7     {
8         sum+=i;
9         if (sum==k)
10        {
11            sum-=1;
12        }
13    }
14    printf("%ld",sum%1000000007);
15    return 0;
16 }
```

	Input	Expected	Got	
✓	2	3	3	✓
✓	2	2	2	✓
✓	3	5	5	✓